# Folha de consulta — Biblioteca padrão do C++

CC0058 · Avaliação presencial · 04/11/2026
Único material de consulta permitido

## Cabeçalho, tipos e entrada/saída

| | |
|---|---|
| `#include <bits/stdc++.h>`<br>`using namespace std;` | Inclui toda a biblioteca padrão. |
| `ios::sync_with_stdio(false);`<br>`cin.tie(nullptr);` | Acelera cin/cout. Depois disso, não misturar com scanf/printf. |
| `getline(cin, s);` | Lê uma linha inteira (com espaços). Após `cin >> x`, usar `cin.ignore()` antes. |
| `while (cin >> x) { ... }` | Lê até o fim da entrada. |
| `cout << fixed << setprecision(k) << x;` | Imprime x com k casas decimais (<iomanip>). |
| `'\n'` | Prefira a endl (que força flush e é mais lento). |
| `int / long long` | int: até $\approx 2{,}1\cdot10^{9}$. long long: até $\approx 9{,}2\cdot10^{18}$. |
| `INT_MAX INT_MIN LLONG_MAX LLONG_MIN` | Limites dos tipos (<climits>). |

## vector<T> (<vector>)

| | |
|---|---|
| `vector<T> v;`<br>`vector<T> v(n);`<br>`vector<T> v(n, val);` | Vazio; n elementos com valor padrão; n cópias de val. |
| `vector<vector<int>> m(n, vector<int>(k, 0));` | Matriz n × k preenchida com 0. |
| `v.push_back(x); v.pop_back();` | Insere/remove no fim. **O(1)** amortizado. |
| `v[i] v.front() v.back()` | Acesso. **O(1)**. Sem verificação de limites. |
| `v.size() v.empty() v.clear()` | Tamanho, se está vazio, esvazia. |
| `v.resize(n); v.assign(n, val);` | Redimensiona; substitui o conteúdo por n cópias de val. |
| `v.insert(it, x); v.erase(it);`<br>`v.erase(first, last);` | Insere/remove no meio. **O(n)**. |
| `v.begin() v.end() v.rbegin()`<br>`v.rend()` | Iteradores (end aponta para depois do último). |

## deque<T> (<deque>)

| | |
|---|---|
| `dq.push_front(x); dq.pop_front();`<br>`dq.push_back(x); dq.pop_back();` | Inserção/remoção nas duas pontas. **O(1)**. |
| `dq.front() dq.back() dq[i]` | Acesso. **O(1)**. |

## stack, queue, priority_queue (<stack>, <queue>)

| | |
|---|---|
| `stack<T> st;`<br>`st.push(x); st.pop(); st.top();` | Pilha (LIFO). Tudo **O(1)**. pop() não retorna valor. |
| `queue<T> q;`<br>`q.push(x); q.pop(); q.front();`<br>`q.back();` | Fila (FIFO). Tudo **O(1)**. |
| `priority_queue<T> pq;` | Heap de **máximo**: top() é o maior. |
| `priority_queue<T, vector<T>, greater<T>> pq;` | Heap de **mínimo**: top() é o menor. |
| `pq.push(x); pq.pop(); pq.top();` | push/pop: **O(log n)**. top: **O(1)**. |
| `x.size() x.empty()` | Comum aos três. Nenhum dos três permite percorrer os elementos. |

## set<T> e multiset<T> (<set>)

| | |
|---|---|
| `set<T> s; multiset<T> ms;` | Elementos sempre ordenados. set não repete; multiset repete. |
| `s.insert(x);` | **O(log n)**. Em set, retorna pair<iterador, bool> (bool = inseriu?). |
| `s.erase(x);` | Remove **todas** as cópias de x. Retorna quantas removeu. |
| `s.erase(it);`<br>`ms.erase(ms.find(x));` | Remove só o elemento do iterador (em multiset, remove uma cópia). |
| `s.find(x)` | Iterador para x, ou s.end() se não existe. **O(log n)**. |
| `s.count(x)` | Quantidade de x (0 ou 1 em set). |
| `s.lower_bound(x)`<br>`s.upper_bound(x)` | Primeiro elemento ≥ x; primeiro elemento > x. **O(log n)**. |
| `*s.begin() *s.rbegin()` | Menor e maior elemento. |
| `for (auto x : s) { ... }` | Percorre em ordem crescente. |

Atenção: use `s.lower_bound(x)` (O(log n)), e não `lower_bound(s.begin(), s.end(), x)` (O(n) em set).

## map<K,V> e multimap<K,V> (<map>)

| | |
|---|---|
| `map<K, V> m;` | Pares chave → valor, ordenados pela chave. Chaves não se repetem. |
| `m[k]` | Acessa o valor de k; **cria** k com valor padrão (0, "") se não existir. |
| `m.count(k) m.find(k)` | Testa existência sem criar. `m.find(k)->second` é o valor. |
| `m.erase(k);` | Remove a chave k. **O(log n)**. |
| `m.lower_bound(k) m.upper_bound(k)` | Como em set, comparando pelas chaves. |
| `for (auto& [k, v] : m) { ... }` | Percorre em ordem crescente de chave (C++17). |
| `it->first it->second` | Chave e valor a partir de um iterador. |

## unordered_set / unordered_map (<unordered_set>, <unordered_map>)

| | |
|---|---|
| `unordered_set<T> us;`<br>`unordered_map<K, V> um;` | Mesma interface de set/map (insert, erase, find, count, []). |
| – | **O(1)** em média, mas **sem ordem** e sem lower_bound/upper_bound. |

## pair e tuple (<utility>, <tuple>)

| | |
|---|---|
| `pair<A, B> p = {a, b};`<br>`make_pair(a, b)` | Par de valores. |
| `p.first p.second` | Componentes do par. |
| `tuple<A, B, C> t = {a, b, c};`<br>`get<0>(t)` | Tupla; get<i> acessa o i-ésimo campo. |
| `auto [a, b] = p;` | Desestrutura par/tupla (C++17). |

Pares e tuplas são comparados lexicograficamente (primeiro campo, depois o segundo...), inclusive em sort, set e priority_queue.

## string (<string>, <cctype>)

| | |
|---|---|
| `s.size() s[i] s.empty()` | Tamanho e acesso. **O(1)**. |
| `s += t; s.push_back(c);`<br>`s.pop_back();` | Concatena; insere/remove caractere no fim. |
| `s.substr(pos, len)` | Substring a partir de pos com até len caracteres. **O(len)**. |
| `s.find(t) s.find(t, pos)`<br>`s.rfind(t)` | Posição da 1ª (ou última) ocorrência, ou string::npos. |
| `s.insert(pos, t); s.erase(pos, len);` | Insere/remove no meio. **O(n)**. |
| `string(n, c)` | String com n cópias do caractere c. |
| `to_string(x) stoi(s) stoll(s)` | Número → string; string → int/long long. |
| `isdigit(c) isalpha(c) isupper(c)`<br>`islower(c) isspace(c)` | Classificação de caracteres. |
| `tolower(c) toupper(c)` | Conversão de caixa (retorna int; use char(...)). |
| `c - 'a' c - '0'` | Índice da letra (0–25); valor do dígito (0–9). |
| `s < t s == t` | Comparação lexicográfica. |

## <numeric>, <cmath> e utilidades

| | |
|---|---|
| `accumulate(first, last, init)` | Soma; o tipo de init define o tipo do resultado (use 0LL). |
| `partial_sum(first, last, out)` | Escreve somas acumuladas a partir de out. |
| `iota(first, last, val);` | Preenche com val, val+1, val+2, .... |
| `gcd(a, b) lcm(a, b)` | MDC e MMC (C++17). |
| `abs(x) sqrt(x) pow(a, b)`<br>`floor(x) ceil(x) log2(x)` | pow, sqrt etc. retornam double (cuidado com precisão). |
| `next(it) prev(it) advance(it, k)`<br>`distance(a, b)` | Navegação por iteradores (<iterator>). |
| `__builtin_popcount(x)`<br>`__builtin_popcountll(x)` | Número de bits 1 de int / long long (GCC). |

| <algorithm> (intervalo [first, last)) | |
|---|---|
| `sort(v.begin(), v.end());` | Ordena crescente. **O(n log n)**. |
| `sort(v.begin(), v.end(), greater<T>());` | Ordena decrescente. |
| `sort(v.begin(), v.end(), [](const T& a, const T& b) { return ...; });` | Comparador próprio: retorna `true` se a deve vir **antes** de b. Deve ser estrito (nunca `true` para a `==` b). |
| `stable_sort(first, last [, cmp]);` | Como `sort`, mas preserva a ordem dos iguais. |
| `reverse(first, last);` | Inverte. **O(n)**. |
| `lower_bound(first, last, x)` `upper_bound(first, last, x)` | Em intervalo **ordenado**: 1º ≥ x; 1º > x. **O(log n)**. Retorna iterador. |
| `it - v.begin()` | Converte iterador em índice. |
| `binary_search(first, last, x)` | `true` se x está no intervalo ordenado. |
| `min(a, b) max(a, b) min({a, b, c})` | Mínimo/máximo (mesmo tipo nos argumentos). |
| `min_element(first, last)` `max_element(first, last)` | Iterador para o menor/maior. Use * para o valor. |
| `count(first, last, x)` `find(first, last, x)` | Ocorrências de x; iterador para o 1º x (ou `last`). **O(n)**. |
| `fill(first, last, x);` | Atribui x a todo o intervalo. |
| `v.erase(unique(v.begin(), v.end()), v.end());` | Remove duplicatas **consecutivas** (ordene antes). |
| `next_permutation(first, last)` `prev_permutation(first, last)` | Próxima/anterior permutação lexicográfica. Retorna `false` ao dar a volta. |
| `swap(a, b);` | Troca dois valores. |